

Introduction to Programming

CSC1103 &1104

Lecture-7

American International University Bangladesh
Dept. of Computer Science
Faculty of Science and Information Technology

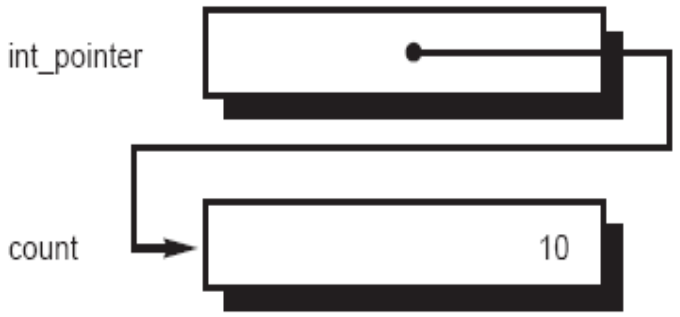
Outline

Pointers

- Pointers and Addresses
- Pointers and Function Arguments
- Pointers and Arrays
- Pointer Arithmetics
- Pointers and strings
- Dynamic memory allocation
- Pointer arrays. Pointers to pointers
- Multidimensional arrays and pointers
- Structures and pointers

Pointers and addresses

- a *pointer* is a *variable* whose *value* is a *memory address*
- `int count = 10;`
- `int *int_pointer;`
- `int_pointer = &count;`
- The **address operator** has the effect of assigning to the variable `int_pointer`, not the value of `count`, but a *pointer* to the variable `count`.
- We say that `int_ptr` "points to" `count`
- The values and the format of the numbers representing memory addresses depend on the computer architecture and operating system. In order to have a portable way of representing memory addresses, we need a different type than integer !
- To print addresses: `%p`



Lvalues and Rvalues

- There are two “values” associated with any variable:
  - An "lvalue" (left value) of a variable is the value of its address, where it is stored in memory.
  - The "rvalue" (right value) of a variable is the value stored in that variable (at that address).
- The lvalue is the value permitted on the left side of the assignment operator '=' (the address where the result of evaluation of the right side will be stored).
- The rvalue is that which is on the right side of the assignment statement
- a=a+1

Declaring pointer variables

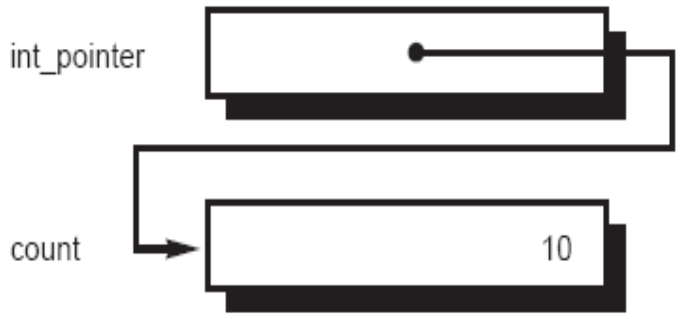
```
type *  variable_name;
```

- it is not enough to say that a variable is a pointer. You also have to specify the *type of variable to which the pointer points !*
  - `int * p1; // p1 points to an integer`
  - `float *  p2; // p2 points to a float`
- Exception: generic pointers `(void *)` indicate that the pointed data type is unknown
  - may be used with explicit type cast to any type (type *)
  - `void * p;`

Indirection (dereferencing) operator *

- To reference the contents of count through the pointer variable int_pointer, you use the ***indirection* operator**, which is the asterisk * as an unary prefix operator. `*int_pointer`
- If a pointer variable `p` has the type `t*`, then the expression `*p` has the type `t`

```
// Program to illustrate pointers
#include <stdio.h>
int main (void)
{
    int count = 10, x;
    int *int_pointer;
    int_pointer = &count;
    x = *int_pointer;
    printf ("count = %i, x = %i\n", count, x);
    return 0;
}
```



Example: pointers

```
// Program to illustrate pointers
#include <stdio.h>
int main (void)
{
    int count = 10;
    int *ip;
    ip = &count;
    printf ("count = %i, *ip = %i\n", count, *ip);
    *ip=4;
    printf ("count = %i, *ip = %i\n", count, *ip);
    return 0;
}
```

Using pointer variables

- The value of a pointer in C is meaningless until it is set pointing to something !

```
int *p;
*p = 4;
```

Severe runtime error !!! **the value 4 is stored in the location to which p points. But p, being uninitialized, has a random value, so we cannot know where the 4 will be stored !**

- How to set pointer values:
  - Using the address operator

```
int *p;
int  x;
p = &x;
*p = 4;
```

  - Using directly assignements between pointer variables

```
int *p;
int *p1;
int  x;
p1 = &x;
p = p1;
*p = 4;
```

NULL pointers

- Values of a pointer variable:
  - Usually the value of a pointer variable is a pointer to some other variable
  - Another value a pointer may have: it may be set to a *null pointer*
- A *null pointer* is a special pointer value that is known not to point anywhere.
- No other valid pointer, to any other variable, will ever compare equal to a null pointer !
- Predefined constant NULL, defined in <stdio.h>
- Good practice: test for a null pointer before inspecting the value pointed !

```
#include <stdio.h>

int *ip = NULL;

if(ip != NULL)  printf("%d\n", *ip);

if(ip )  printf("%d\n", *ip);
```

`const` and pointers

- **With pointers, there are two things to consider:**
  - **whether the pointer will be changed**
  - **whether the value that the pointer points to will be changed.**
- Assume the following declarations:

```
char c = 'X';
char *charPtr = &c;
```

- If the pointer variable is always set pointing to c, it can be declared as a const pointer as follows:

```
char * const charPtr = &c;
*charPtr = 'Y'; // this is valid
charPtr = &d;   // not valid !!!
```

- If the location pointed to by charPtr will not change *through the pointer variable charPtr*, that can be noted with a declaration as follows:

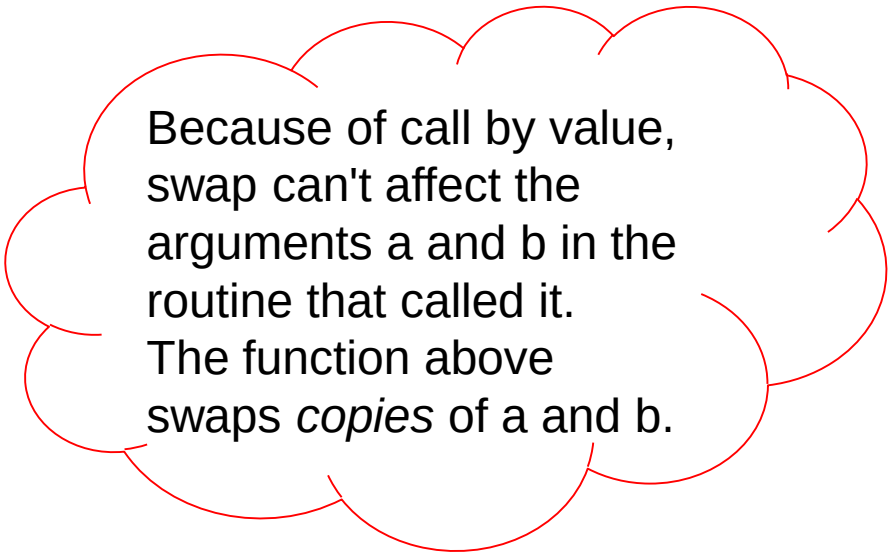
```
const char *charPtr = &c;
charPtr = &d;   // this is valid
*charPtr = 'Y'; // not valid !!!
```

Pointers and Function Arguments

- Recall that the C language passes arguments to functions by value (except arrays)
- there is no direct way for the called function to alter a variable in the calling function.

```
void swap(int x, int y)   /* WRONG */
    {
        int temp;
        temp = x;
        x = y;
        y = temp;
    }

swap(a,b);
```

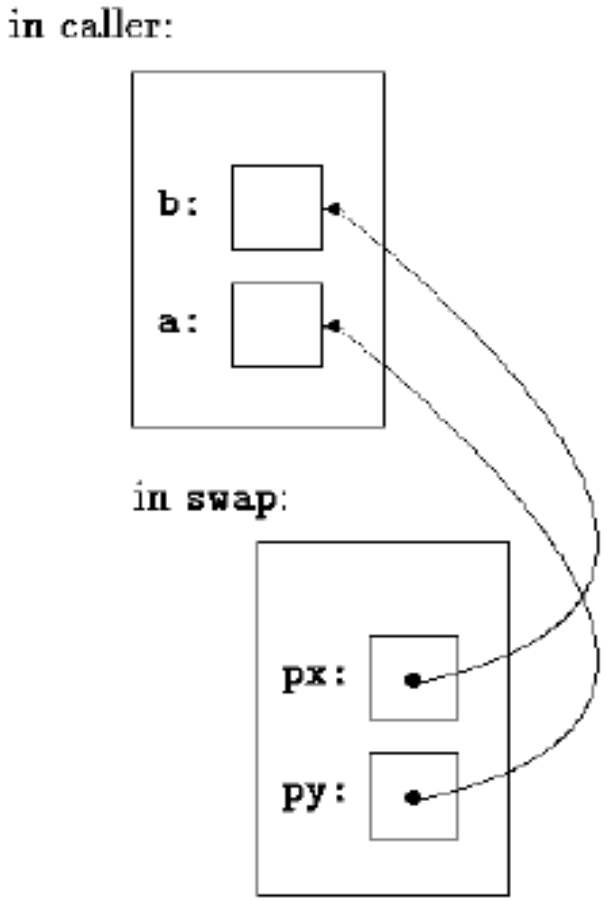


Pointers and Function Arguments

- *If it is necessary that a function alters its arguments, the caller can pass pointers to the values to be changed*
- Pointer arguments enable a function to access and ***change*** variables in the function that called it

```
void swap(int *px, int *py)
/* interchange *px and *py */
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}


int a=3, b=5;
swap(&a, &b);
```



Pointers and arrays

- In C, there is a strong relationship between pointers and arrays
- Any operation that can be achieved by array subscripting can also be done with pointers

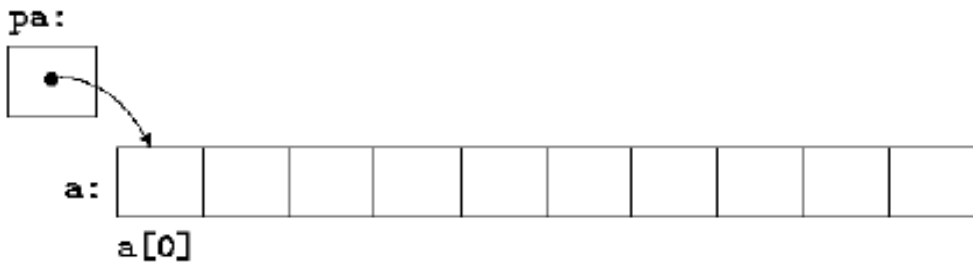
```
int a[10];
```




```
int *pa;
pa=&a[0];
```

Or

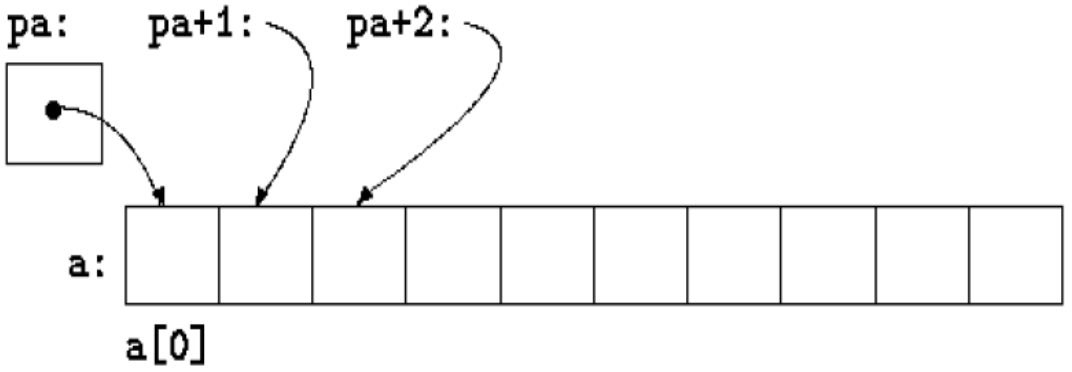
```
pa=a;
```



The value of a variable of type array is the address of element zero of the array.
The name of an array is a synonym for the location of the initial element.

Pointers and Arrays

- If pa points to a particular element of an array, then by definition pa+1 points to the next element, pa+i points i elements after pa, and pa-i points i elements before.
- If pa points to a[0], *(pa+1) refers to the contents of a[1], pa+i is the address of a[i], and *(pa+i) is the contents of a[i].
- The value in a[i] can also be written as *(a+i). The address &a[i] and a+i are also identical
- These remarks are true regardless of the type or size of the variables in the array a !



Arrays are **constant** pointers

```
int a[10];
int *pa;

pa=a;

pa++;
```

OK. Pointers are variables that can be assigned or incremented

```
int a[10];
int *pa;

a=pa;

a++;
```

Errors !!!

The name of an array is a CONSTANT having as a value the location of the first element.
You cannot change the address where the array is stored !
An array's name is equivalent with a ***constant*** pointer

Arrays as parameters

- When an array name is passed to a function, what is passed is the location of the first element. Within the called function, this argument is a local variable, and so an array name parameter is a pointer, that is, a variable containing an address.
- As formal parameters in a function definition, T `s[]` and T `*s` are equivalent, for any type T; The latter is preferred because it says more explicitly that the variable is a pointer.
- Examples:
- f(int arr[]) { ... } is equivalent with f(int *arr) { ... }
- f(char s[]) { …} is equivalent with f(char *s) { …}

Example: Arrays as parameters

```
void print1(int tab[], int N) {
    int i;
    for (i=0; i<N; i++)
        printf("%d ",tab[i]);
}
void print2(int tab[],int N) {
    int * ptr;
    for (ptr=tab; ptr<tab+N; ptr++)
        printf("%d ", *ptr);
}
void print3(int *tab,int N) {
    int * ptr;
    for (ptr=tab; ptr<tab+N; ptr++)
        printf("%d ", *ptr);
}
void print4(int *tab,int N) {
    int i;
    for (i=0; i<N; i++, tab++)
        printf("%d ", *tab);
}
```

The formal parameter can be declared as array or pointer !
In the body of the function, the array elements can be accessed through indexes or pointers !

```
void main(void) {
    int a[5]={1,2,3,4,5};
    print1(a,5);
    print2(a,5);
    print3(a,5);
    print4(a,5);
}
```

Example: Arrays as parameters

```
/* strlen: return length of string s */
int strlen(char *s)
{
    int n;
    for (n = 0; *s != '\0'; s++)
        n++;
    return n;
}
```

The actual parameter can be declared as array or pointer !

```
char array[100]="Hello, world";
char *ptr="Hello, world";

strlen("Hello, world"); /* string constant */
strlen(array); /* char array[100]; */
strlen(ptr);   /* char *ptr; */
```

Example: Arrays as parameters

```
int strlen(char *s)
{
    if (*s=='\0')
       return 0;
    else
         return 1 + strlen(++s);
}
```

The recursive call gets as parameter the subarray starting with the second element

It is possible to pass part of an array to a function, by passing a pointer to the beginning of the subarray.
For example, if a is an array, f(&a[2]) and f(a+2) both pass to the function f the address of the subarray that starts at a[2].

Pointer arithmetic

- All operations on pointers take into account the size of the pointed type (sizeof(T)) !
- *Valid pointer operations:*
  - Assignement between pointers of the same type
  - Addition/ subtraction between a pointer and an integer
  - Comparison between two pointers that point to elements of the same array
  - Subtraction between two pointers that point to elements of the same array
  - Assignment or comparison with zero (NULL)

Pointer arithmetic

- **Increment/decrement:** if *p is a pointer to type T, p++ increases the value of p* by *sizeof(T) (sizeof(T) is the amount of storage needed for an object of type T).* Similarly, *p--* decreases *p* by *sizeof(T);*

```
T tab[N];
T * p;
int i;
p=&tab[i];
p++;   // p contains the address of tab[i+1];
```

- **Addition/subtraction with an integer**: if *p is a pointer to type T* and *n* an integer, *p+n increases the value of p* by *n*sizeof(T).* Similarly, *p-n* decreases *p* by *n*sizeof(T);*

```
T tab[N];
T * p;
p=tab;
p=p+n;   // p contains the address of tab[n].
```

Pointer arithmetic

- **Comparison of two pointers.**
- ***If p and q point to members of the same array***, then relations like ==, !=, <, >=, etc., work properly.
  - For example, p < q is true if p points to an earlier element of the array than q does.
- Any pointer can be meaningfully compared for equality or inequality with zero.
- **Pointer subtraction** :
- ***if p and q point to elements of the same array,*** and p<q, then q-p+1 is the number of elements from p to q inclusive.

- The behavior is undefined for arithmetic or comparisons with pointers that do not point to members of the same array.

Example: pointer subtraction

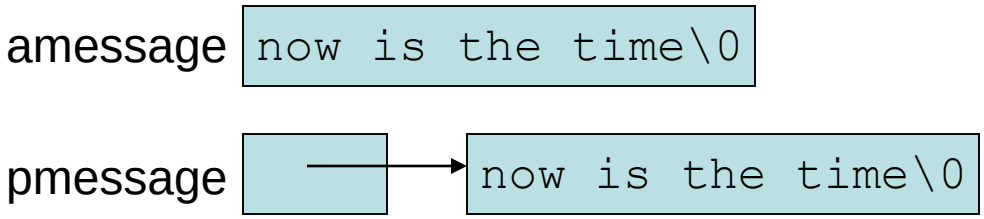
```
/* strlen: return length of string s */
int strlen(char *s)
{
    char *p = s;
    while (*p != '\0')
        p++;
    return p - s;
}
```

Character pointers

- A *string constant:* "I am a string"
- is an array of characters. In the internal representation, the array is terminated with the null character '\0' so that programs can find the end. The length in storage is thus one more than the number of characters between the double quotes.
- a string constant is accessed by a pointer to its first element
- `char *pmessage;`
- `pmessage = "now is the time";`
- assigns to pmessage a pointer to the character array. This is *not* a string copy; only pointers are involved !

Character pointers

```
char amessage[] = "now is the time"; /* an array */
char *pmessage = "now is the time"; /* a pointer */
```



- `amessage` is an array, just big enough to hold the sequence of characters and '\0' that initializes it. Individual characters within the array may be changed but amessage will always refer to the same storage.
- `pmessage` is a pointer, initialized to point to a string constant; the pointer may subsequently be modified to point elsewhere, but the result is undefined if you try to modify the string contents.

Precedence of operators

- **`*p++`** increments p after fetching the character that p points to

```
char *p=“hello” ;
printf(“%c”,*p++); // displays h
```

- **`*++p`** increments p before fetching the character that p points to

```
char *p=“hello” ;
printf(“%c”,*++p); // displays e
```

Example: Character pointers and functions

```
/* strcpy: copy t to s; array subscript version */
void strcpy(char s[], char t[])
{
    int i;
    i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}

/* strcpy: copy t to s; pointer version */
void strcpy(char *s, char *t)
{
    int i;
    i = 0;
    while ((*s = *t) != '\0') {
        s++;
        t++;
    }
}
```

Example: Character pointers and functions

```
/* strcpy: copy t to s; pointer version 2 */
void strcpy(char *s, char *t)
{
    while ((*s++ = *t++) != '\0')
    ;
}

/* strcpy: copy t to s; pointer version 3 */
void strcpy(char *s, char *t)
{
    while (*s++ = *t++)
    ;
}
```

Exercise: What is the output ?

```
void swap_vectors(int *a, int *b, int n) {
    int *t;
    t=a;
    a=b;
    b=t;
    print_vector(a,3);
    print_vector(b,3);
    }
...
int a[3]={1,2,3};
int b[3]={4,5,6};
swap_vectors(a,b,3);
print_vector(a,3);
print_vector(b,3);
```

Exercise: different versions of adding 2 vectors

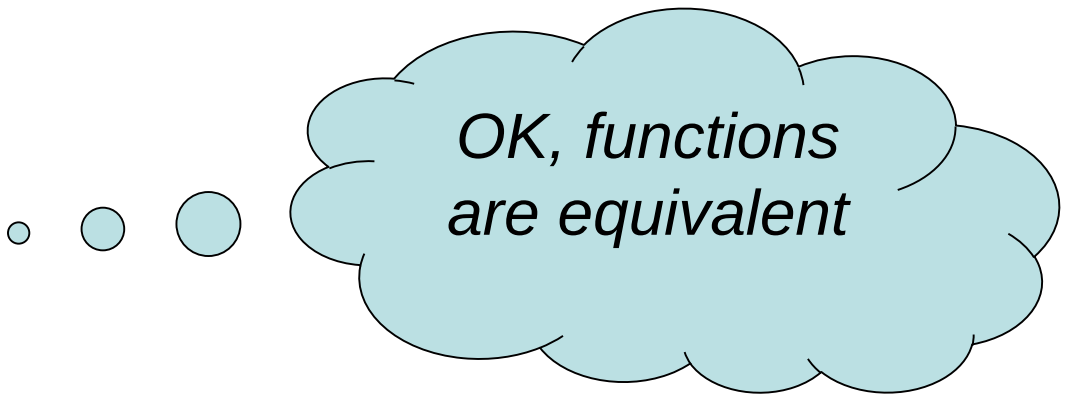
```
void add_vectors1 (int a[], int b[], int r[], int n) {
        int i;
        for (i=0; i<n; i++)
            r[i]=a[i]+b[i];
            }

void add_vectors2 (int *a, int *b, int *r, int n) {
        int i;
        for (i=0; i<n;  a++, b++, r++, i++)
            *r=*a+*b;
            }
```

Exercise: different versions of adding 2 vectors

```
int  a[3]={1,2,3};
 int b[3]={4,5,6};
 int r[3];

add_vectors1(a,b,r,3);
print_vector(r,3);
add_vectors2(a,b,r,3);
print_vector(r,3);
```



Exercise: different versions of adding 2 vectors

```
int * add_vector3(int *a, int *b, int n) {
 int r[3];
 int i;
 for (i=0; i<n; i++, a++, b++)
        r[i]=*a+*b;
  return r;
  }

 …
int a[3]={1,2,3};
int b[3]={4,5,6};
int * rr;
rr=add_vector3(a,b,3);
print_vector(rr,3);
```

NO !!!

BIG MISTAKE *!*
The function returns the address of a local variable !

Exercise: different versions of adding 2 vectors

```
int * add_vector3(int *a, int *b, int n) {
 static  int r[3];
  int i;
  for (i=0; i<n; i++, a++, b++)
          r[i]=*a+*b;
   return r;
   }


  …
int a[3]={1,2,3};
int b[3]={4,5,6};
int * rr;
rr=add_vector3(a,b,3);
print_vector(rr,3);
```

Does this solve the problem ?

Exercise: different versions of adding 2 vectors

```
int * add_vector3(int *a, int *b, int n) {
 static  int r[3];
  int i;
  for (i=0; i<n; i++, a++, b++)
          r[i]=*a+*b;
   return r;
   }


  …
 int a[3]={1,2,3};
 int b[3]={4,5,6};
 int c[3]={0,1,2};
 int d[3]={3,4,5};
 int * rr1, *rr2;
 rr1=add_vector3(a,b,3);
 rr2=add_vector3(c,d,3);
 print_vector(rr1,3);
 print_vector(rr2,3);
```

Does this solve the problem ?

NO !

Dynamic memory allocation

- Variable definitions in C: the C compiler automatically allocates the correct amount of storage (1-n memory locations) where the variable will be stored -> this happens before the execution of the actual program starts
- Sometimes it can be useful to *dynamically* allocate storage while a program is running:
- Suppose we have a program that is designed to read in a set of data from input into an array in memory but we don't know how many data items there are until the program starts execution. We have three choices:
  - Define the array to contain the maximum number of possible elements at compile time.
  - Use a variable-length array to dimension the size of the array at runtime.
  - Allocate the array dynamically using one of C's memory allocation routines.

malloc()

- <stdlib.h>
- `void * malloc(int n);`
- malloc allocates *n* bytes of memory and returns a pointer to them if the allocation was succesful, NULL otherwise
- Before using the pointer returned by malloc, it has to be checked if it is not NULL !!
- The pointer returned by malloc is of the generic type void *; it has to be converted to a concrete pointer type

malloc() Example1

```
#include <stdlib.h>

…

char *line;
int linelen;

printf (“How long is your line ?”);
scanf(“%d”,&linelen);

line = (char *) malloc(linelen);

/* incomplete here - malloc's return value not checked */

getline(line, linelen);
```

malloc() Example 2

```
char *somestring=“how are you”;
char *copy;
...
copy = (char *) malloc(strlen(somestring) + 1);

/* incomplete -- malloc's return value not checked */

strcpy(copy, somestring);
```

Checking what malloc returns !

- When malloc is unable to allocate the requested memory, it returns a *null pointer*. Therefore, whenever you call malloc, it's vital to check the returned pointer before using it !

```
char * line = (char *) malloc(linelen); ;
if(line == NULL) {
        printf("out of memory\n");
        return; // exits current function
}
```

```
char * line = (char *) malloc(linelen);
if(line == NULL) {
        printf("out of memory\n");
        exit(1); // exits all nested function calls,
                 //  terminates program
}
```

Dynamic allocation of arrays

- Dynamic allocation of an array of N elements of type TIP:
- `TIP * p;`
- `p= (TIP *) malloc(N*sizeof(TIP));`
- Pointer p will point to a memory block big enough to hold N elements of type TIP.
- Variable p can be used in the same way as if it was declared:
- `TIP p[N];`

Example: dynamic allocation of arrays

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int n;
    int * tab;
    int i;
    printf("Input number of elements: \n");
    scanf("%d", &n);
    if ((tab=(int *)malloc(n * sizeof(int)))==NULL) {
        printf(“Memory allocation error !\n");
        exit(1);
    }
    for (i=0; i<n; i++)
        scanf("%d", &tab[i]);
    for (i=0; i<n; i++)
        printf("%d ", tab[i]);
    free(tab);
    return 1;
}
```

Example: dynamic allocation of character strings

```
void main(void)
{
    char sir1[40];
    char *sir2;
    printf(“Enter a string: \n");
    scanf("%40s", sir1);
    if ((sir2=(char *)malloc(strlen(sir1)+1))==NULL) {
        printf(“memory allocation error !\n");
        exit(1);
        }
    strcpy(sir2, sir1);
    printf(“The copy is: %s \n", sir2);
    ..........
}
```

Lifetime of dynamic allocated memory

- Memory allocated with malloc lasts as long as you want it to. It does not automatically disappear when a function returns, as automatic variables do:

```
void fct(void) {
        int *p;
        p=(int*) malloc(10*sizeof(int));
        return;
}
```



The memory area allocated here remains occupied also after the function call is ended ! Only the pointer variable p accessing it disappears.

Lifetime example

- Have a function create and return an array of values:

```
int * fct(void) {
       int *p;
       p=(int*) malloc(10*sizeof(int));
       // fill p with values . . .
       return p;
}
```

YES !

```
int * fct(void) {
       int p[10];
       // fill p with values . . .
       return p;
}
```

NO !

Exercise: different versions of adding 2 vectors

```
int * add_vector4(int *a, int *b, int n) {
 int * r;
 r=(int *) malloc(sizeof (int) * n);
 if (r==NULL) exit(1);
  int i;
  for (i=0; i<n; i++, a++, b++)
          r[i]=*a+*b;
   return r;
   }


  …
int a[3]={1,2,3};
int b[3]={4,5,6};
int * rr;
rr=add_vector4(a,b,3);
print_vector(rr,3);
```

free()

- Dynamically allocated memory is deallocated with the free function. If p contains a pointer previously returned by malloc, you can call
- `free(p);`
- which will ``give the memory back'' to the heap of memory from which malloc requests are satisfied.
- the memory you give back by calling free() is immediately usable by other parts of your program. (Theoretically, it may even be usable by other programs.)
- When your program exits, any memory which it has allocated but not freed should be automatically released by the operating system.
- Once you've freed some memory you must remember not to use it any more. After calling `free(p)` it is probably the case that p still points at the same memory. However, since we've given it back, it's now ``available,'' and a later call to `malloc` might give that memory to some other part of your program.

Structures and pointers

- Structure pointers are just like pointers to ordinary variables. The declaration
- ```
  struct point *pp;
  ```
- says that pp is a pointer to a structure of type struct point. If pp points to a point structure, *pp is the structure, and (*pp).x and (*pp).y are the members.
- To use pp:
- ```
  struct point origin, *pp;
  ```
- ```
  pp = &origin;
  ```
- ```
  printf("origin is (%d,%d)\n", (*pp).x, (*pp).y);
  ```
- The parentheses are necessary in (*pp).x because the precedence of the structure member operator . is higher then *. The expression *pp.x means *(pp.x), which is illegal here because x is not a pointer.

Pointers to structures

- Pointers to structures are so frequently used that an alternative notation is provided as a shorthand. If p is a pointer to a structure, then referring to a particular member can be done by:
- p->*member-of-structure*
- Equivalent with `(*p).member-of-structure`
- `printf("origin is (%d,%d)\n", pp->x, pp->y);`
- Both . and -> associate from left to right, so if we have
- `struct rect r, *rp = &r;`
- then these four expressions are equivalent:

```
r.pt1.x
rp->pt1.x
(r.pt1).x
(rp->pt1).x
```

Precedence of operators

- The structure operators . and ->, together with () for function calls and [] for subscripts, are at the top of the precedence hierarchy and thus bind very tightly.
- For example, given the declaration
- ```
  struct {
        int len;
        char *str;
    } *p;
  ```
- then `++p->len` increments len, not p, because the implied parenthesization is `++(p->len)` .
- Parentheses can be used to alter binding: (++p)->len increments p before accessing len, and (p++)->len increments p afterward. (This last set of parentheses is unnecessary.
- `*p->str` fetches whatever str points to;
- `*p->str++` increments str after accessing whatever it points to (just like *s++)
- `(*p->str)++` increments whatever str points to
- `*p++->str` increments p after accessing whatever str points to.
  ```

Pointer arrays. Pointers to Pointers

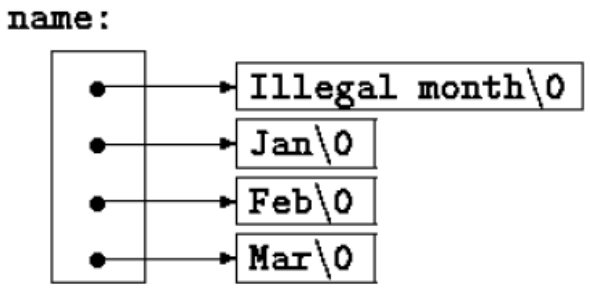
- Since pointers are variables themselves, they can be stored in arrays just as other variables can

Pointers vs Multidimensional arrays

- `int a[10][20];`
- `int *b[10];`
- a[3][4] and b[3][4] are both syntactically legal references to an int.
- a is a true two-dimensional array: 200 int-sized locations have been set aside, and the conventional rectangular subscript calculation 20 * *row* +*col* is used to find the element a[row,col].
- For b, however, the definition only allocates 10 pointers and does not initialize them; initialization must be done explicitly, either statically or with code. Assuming that each element of b does point to a twenty-element array, then there will be 200 ints set aside, plus ten cells for the pointers. The important advantage of the pointer array is that the rows of the array may be of different lengths. That is, each element of b need not point to a 20 element vector; some may point to two elements, some to fifty, and some to none at all.

Pointers vs Multidimens arrays

- char *name[] = { "Illegal month", "Jan", "Feb", "Mar" };
- char aname[][15] = { "Illegal month", "Jan", "Feb", "Mar" };



aname:

Illegal month\0 Jan\0	Feb\0	Mar\0
0 15	30	45

Program design example

- a program that will read an then sort a set of text lines into alphabetic order
- Requirements:
  - Program reads a number of lines, until EOF, but not more than MAXLINES=5000
  - Each line may have a different number of characters, but not more than MAXLEN=1000
  - Program sorts lines in alphabetic order
  - Program displays sorted lines
- How to approach program design problems:
  - Data structures
  - Algorithms
  - Functions

Recall: Sorting algorithm

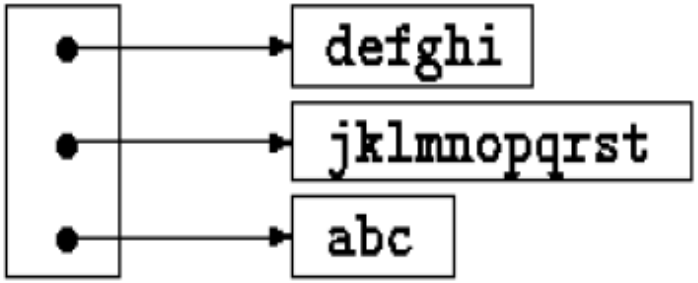
```
// sort an array of integers into ascending order

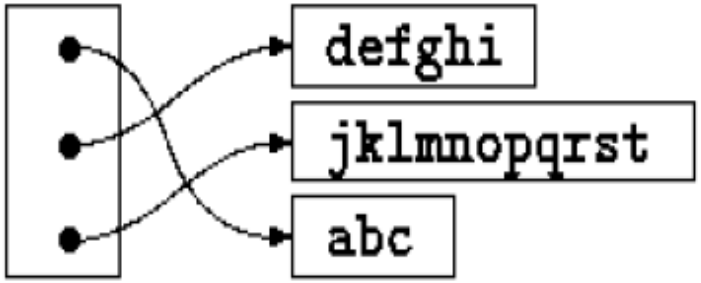
void sort (int a[], int n) {
    int i, j, temp;
    for ( i = 0; i < n - 1; ++i )
        for ( j = i + 1; j < n; ++j )
            if ( a[i] > a[j] ) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
}
```

The sorting algorithms will work, except that now we have to deal with lines of text, which are of different lengths, and which, unlike integers, can't be compared or moved in a single operation (strcmp, strcpy).
We need a data representation that will cope efficiently and conveniently with variable-length text lines.

Data structure: array of pointers

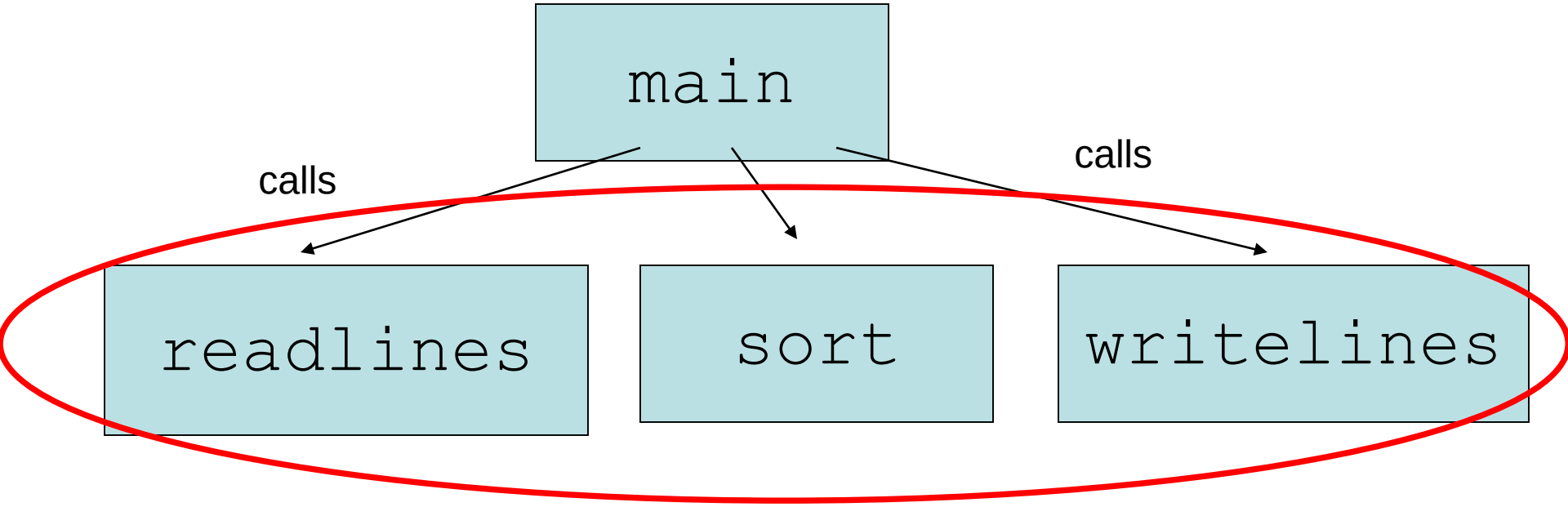
- Each line can be accessed by a pointer to its first character. The pointers themselves can bee stored in an array.
- Two lines can be compared by passing their pointers to `strcmp`.
- When two out-of-order lines have to be exchanged, the pointers in the pointer array are exchanged, not the text lines themselves.






```
#define MAXLINES 5000     /* max number of lines to be sorted */
char *lineptr[MAXLINES];  /* array of pointers to text lines */
```

Top-down program refinement



1. Split problem (program) in subproblems (functions)
2. For each function:
   - Functionality (what it does)
   - Prototype
   - Contract: how exactly does it handle input arguments, which are its returned results and side-effects (on global variables)

Designing the functions

- Readlines:
  - `int readlines(char *lineptr[], int nlines);`
  - Argument `lineptr`: array will be filled with pointers to the read lines
  - Argument `nlines`: maximum number of lines that can be read
  - Return value: number of lines that has been read or -1 if an error occured
- Writelines:
  - `void writelines(char *lineptr[], int nlines);`
  - Writes `nlines` elements of array `lineptr`
- Sort
  - `void sort(char *lineptr[], int nlines);`

Example: main program

```
#include <stdio.h>
#include <string.h>

#define MAXLINES 5000     /* max #lines to be sorted */
char *lineptr[MAXLINES];  /* pointers to text lines */

int readlines(char *lineptr[], int nlines);
void writelines(char *lineptr[], int nlines);
void sort(char *lineptr[], int nlines);

int main(void) {
      int nlines;     /* number of input lines read */
      if ((nlines = readlines(lineptr, MAXLINES)) >= 0) {
          sort(lineptr,  nlines);
           writelines(lineptr, nlines);
          return 0;
      } else {
          printf("error: input too big to sort\n");
          return 1;
      }
  }
```

Implementing the functions

- If a function corresponds to a problem which is still complex, split it in other subproblems (functions)
- Top-down decomposition
- Stepwise refinement

Example: decomposing `readlines()`

- Readlines:
  - repeatedly: reads a line from stdin, allocates a buffer of the exact line length and copies there the input, stores pointer to buffer in current element of lineptr

- int getline(char *s, int lim);
  - Reads a maximum of lim-1 characters from stdin, until \n or EOF
  - Argument s must point to an array that is able to hold a maximum of lim characters
  - Returns the number of characters read
  - The characters read will be stored in array s; character \n is also included. Null character is appended.

Example: `readlines()` implementation

```
#define MAXLEN 1000  /* max length of any input line */
int getline(char *, int);

/* readlines:  read input lines */
int readlines(char *lineptr[], int maxlines)   {
    int len, nlines;
    char *p, line[MAXLEN];
    nlines = 0;
    while ((len = getline(line, MAXLEN)) > 0)
        if (nlines >= maxlines ||
             ((p = (char *)malloc(len)) == NULL))
            return -1;
        else {
            line[len-1] = '\0';  /* delete newline */
            strcpy(p, line);
            lineptr[nlines++] = p;
        }
    return nlines;
}
```

Example: getline

```
/* getline:  read a line into s, return length  */
   int getline(char s[],int lim)
   {
       int c, i;
       for (i=0; i < lim-1 &&
                    (c=getchar())!=EOF && c!='\n'; ++i)
           s[i] = c;
       if (c == '\n') {
           s[i] = c;
           ++i;
       }
       s[i] = '\0';
       return i;
   }
```

Example: writelines

```
/* writelines:  write output lines */
   void writelines(char *lineptr[], int nlines)
   {
       int i;
       for (i = 0; i < nlines; i++)
           printf("%s\n", lineptr[i]);
   }
```

Example: sort

```
void sort (char* v[], int n)
{
int i, j;
char *temp;
for ( i = 0; i < n - 1; ++i )
        for ( j = i + 1; j < n; ++j )
                if ( strcmp(v[i], v[j])>0 ) {
                /* swaps only pointers to strings */
                        temp = v[i];
                        v[i] = v[j];
                        v[j] = temp;
                }
}
```